

INFO145 - Diseño y Análisis de Algoritmos

11 - Algoritmos Greedy

Héctor Ferrada
académico

Instituto de Informática
Universidad Austral de Chile

Algoritmos Greedy

Son también llamados algoritmos **Aváros** o **Voraces** y se caracterizan por:

- No esperan o no dependen de otros pasos para tomar sus decisiones.
- Eligen siempre lo mejor entre lo que tienen en el momento, esperando que estas decisiones, quizá “imprudentes” o “apresuradas”, pero óptimas para ese momento, conlleven a una buena solución final o inclusive al óptimo global.
- No siempre encuentran el óptimo global, pero son más eficientes en cuanto al tiempo de cómputo.
- La premisa es que un algoritmo voráz siempre es muy rápido.

Minimum Spanning Tree (MST)

Dados:

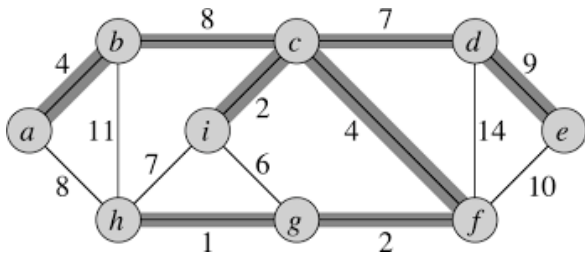
1. Un **grafo no dirigido** $G = (V, E)$; donde V es el conjunto de vértices y E es el conjunto de arcos (u, v) , con $u, v \in V$.
2. Una función de peso, o costo, asociada a los arcos, denotada por:
 $\omega : E \rightarrow \mathbb{R}^+$
 $\omega(u, v)$ indica cuál es el costo de conectar u con v .

Se desea encontrar un subconjunto acíclico $T \subset E$ que conecte a todos los vértices de G , tal que:

$$\omega(T) = \sum_{(u,v) \in T} \omega(u, v) \text{ sea mínimo.}$$

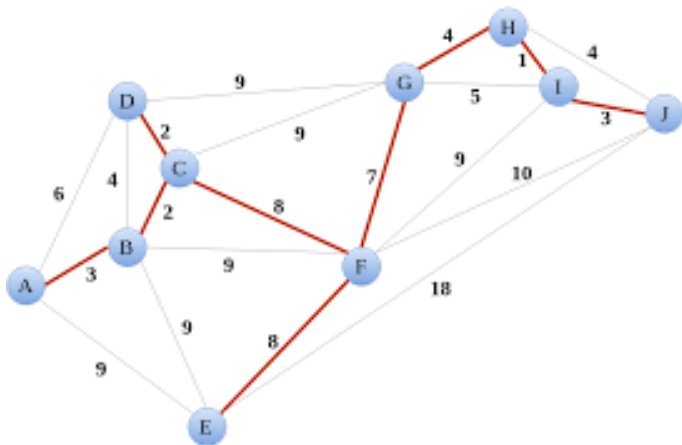
- Cómo T es acíclico $\Rightarrow T$ es un árbol, y por conectar todos los vértices de E es llamado *Spanning Tree* (árbol de expansión).
- El problema de encontrar T se llama *Minimum Spanning Tree*; y note que puede existir más de un MST para G y ω .

Ejemplo de MST



MST obtenido con el algoritmo de Kruskal; donde los arcos sombreados pertenecen al MST encontrado.

Ejemplos MST



En la imagen, los arcos pertenecientes al T que Kruskal obtiene están en rojo.

Algoritmo de Kruskal para encontrar un MST

- Su solución es incremental —añade un arco en cada paso.
- El **Invariante de Iteración** es: previo a cada iteración, la solución parcial construida, hasta ese momento, siempre es subconjunto de algún árbol de expansión mínima.
- Demostraremos luego que Kruskal siempre encuentra el óptimo global para el problema del MST.
- Kruskal cumple en T el siguiente principio para todo MST:
 $\# \text{ aristas} = \# \text{ vértices} - 1$

Algoritmo de Kruskal

Input: El grafo no dirigido $G = (V, E)$ y la función de costo $\omega(\cdot) : E \longrightarrow \mathbb{R}^+$

Output: T que es un MST para G y ω

Kruskal($G = (V, E)$, $\omega(\cdot) : E \longrightarrow \mathbb{R}^+$) {
 $n = |V|$

(1) Ordenar E por $\omega(\cdot)$ ascendentemente

$T = \emptyset$

 while ($|T| < n - 1$) do {

(2) $e = \text{extractMin}(E)$: toma la siguiente menor arista de E

(3) if (e No forma ciclo en T) then

$T = T \cup \{e\}$

 }

 return T

}

Algoritmo de Kruskal

Input: El grafo no dirigido $G = (V, E)$ y la función de costo $\omega(\cdot) : E \rightarrow \mathbb{R}^+$

Output: T que es un MST para G y ω

Kruskal($G = (V, E)$, $\omega(\cdot) : E \rightarrow \mathbb{R}^+$) {
 $n = |V|$

(1) Ordenar E por $\omega(\cdot)$ ascendentemente

$T = \emptyset$

 while ($|T| < n - 1$) do {

(2) $e = \text{extractMin}(E)$: toma la siguiente menor arista de E

(3) if (e No forma ciclo en T) then

$T = T \cup \{e\}$

 }

 return T

}

Daremos solución a las operaciones en azul de la siguiente manera:

(1) Ordenar E por $\omega(\cdot)$ ascendentemente $\Rightarrow$ creamos un minheap $\rightarrow O(|E|)$

(2) $e = \text{extractMin}(E)$ $\Rightarrow$ extraemos desde el heap y hacemos heapify
 $\rightarrow O(\log |E|)$

(3) Comprobar si (e No forma ciclo en T) es algo más complejo, luego veremos la solución de Kruskal & Prim basada en estructuras para UNION-FIND.

Teorema de Kruskal

Teorema. Kruskal obtiene un MST para $G = (V, E)$ y $\omega(\cdot)$.

Demostración. Por inducción sobre $|T|$

$\Rightarrow$ En cada iteración, el T de Kruskal es subconjunto de un MST.

Suponga que Kruskal se equivoca en la iteración que incorpora a la arista $e = (u, v)$, y entonces $T \cup \{e\}$ no es un subconjunto de un MST.

$\Rightarrow$ Existe otro camino que une a u y a v y que si es parte de un MST' ; es decir, $\exists e' \in E$ en este otro mejor camino que conecta a u y v (ver imagen sgte. slide). Entonces $\omega(e') \geq \omega(e)$, ya que e fue obtenido con $extractMin(E)$.

Entonces tenemos dos posibilidades:

i- Si $\omega(e') = \omega(e)$ entonces no hay problema y ambos son un MST.

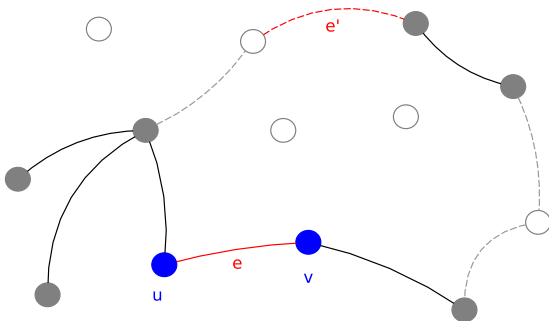
ii- Si $\omega(e') > \omega(e) \Rightarrow$ Si al MST' que contiene e' , le quitamos e' y lo reemplazamos por e , sigue siendo un spanning tree; y además se cumple:

$$\begin{aligned}\omega(MST' - \{e'\} \cup \{e\}) &= \omega(MST') - \omega(\{e'\}) + \omega(\{e\}) \\ &< \omega(MST') \rightarrow \text{Contradicción, ya que } MST' \text{ es MST}\end{aligned}$$

$\therefore MST' - \{e'\} \cup \{e\}$ no puede tener un costo menor que un MST y seguir siendo un MST.

Q.E.D.

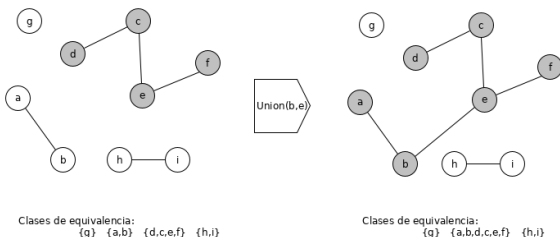
Teorema de Kruskal



En la imagen, los vértices grises ya han sido conectados por etapas iterativas de Kruskal y son parte de algún MST válido para $G = (V, E)$. El supuesto absurdo es que, en la etapa actual, Kruskal se equivoca al seleccionar el arco e para unir u con v , y entonces $T \cup \{e\} \notin$ a algún MST. Esto implica que $\exists e'$, por otra ruta, que une u con v y que si es parte de un MST' , lo que genera una contradicción, ya que (parte *ii-* de la demostración) $\omega(e') > \omega(e)$ y e fue retornado primero que e' en la operación $extractMin(E)$.

Estructuras de datos para UNION-FIND

La operación $\text{UNION}(b, e)$ consiste en unir las **clases de equivalencia**¹ de b y de e :



En el gráfico de la imagen, al resolver $\text{UNION}(b, e)$ el bosque pasa de 4 árboles: $\{g\}$, $\{a, b\}$, $\{h, i\}$ y $\{d, c, e, f\}$ a tres árboles o clases de equivalencia: $\{g\}$, $\{a, b, d, c, e, f\}$ y $\{h, i\}$

¹Las clases de equivalencia son los conjuntos formados por las uniones de todos los vértices de cada árbol en un grafo tipo bosque (acíclico).

Estructuras de datos para UNION-FIND

Una de las estructura más sencillas es la siguiente:

1. En un arreglo $X[1..n]$ almacenamos el ID de la clase de equivalencia a la cual pertenece cada uno de los n elementos.
2. A elección, podemos incluir o no una lista enlazada para cada una de las clases de equivalencia —esto se suele hacer en la práctica para acelerar la operación UNION y no tener que recorrer (necesariamente) todo el arreglo X .
3. Esto nos permite resolver:
 $\text{FIND}(x) \rightarrow O(1)$
 $\text{UNION}(x_i, x_j) \rightarrow O(n)$

Note que los tiempos asintóticos se mantienen aún sin utilizar la lista enlazada.

Algoritmo de Kruskal & Prim

- El algoritmo de Kruskal & Prim hace uso de estructuras de datos para UNION-FIND, las cuales se basan en **clases de equivalencia**.
- Al inicio, todos los vértices están desconectados formando $n = |V|$ árboles en G , cada uno de los árboles de G forma una clase de equivalencia distinta.
- $\exists$ un conjunto C que almacena las clases o representantes para todos los árboles del problema, al inicio $C = \{c_1, c_2, \dots, c_n\}$. Entonces tenemos soporte para las siguientes operaciones:
 - $\text{FIND}(u)$: retorna el ID de clase para el vértice u .
 - $\text{UNION}(u, v)$: une las clases de u y de v en una sola clase.
- Kruskal & Prim finaliza cuando existe un único árbol en el bosque representado por G , es decir que todos los vértices están conectados.

Algoritmo de Kruskal & Prim

Input: El grafo no dirigido $G = (V, E)$ y la función de costo $\omega(\cdot) : E \rightarrow \mathbb{R}^+$

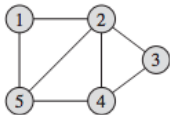
Output: Un MST para G y ω

```
Kruskal-Prim( $G = (V, E)$ ,  $\omega(\cdot) : E \rightarrow \mathbb{R}^+$ ) {  
     $n = |V|$   
    makeMinHeap( $E, \omega(\cdot)$ ) →  $O(|E|)$   
     $T = \emptyset$   
    clases =  $\{\{v_1\}, \{v_2\}, \dots, \{v_n\}\}$   
    while ( $|T| < n - 1$ ) do { →  $O(|E|)$   
         $e = (u, v) = \text{extractMin}(E)$  →  $O(\log |E|)$   
        if ( $\text{FIND}(u) \neq \text{FIND}(v)$ ) then →  $O(1)$  con un array para los IDs  
             $T = T \cup \{e\}$   
            UNION( $u, v$ ) →  $O(|V|)$   
    }  
    return  $T$   
}
```

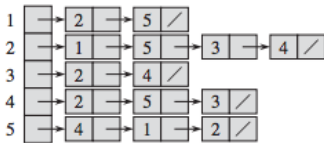
$T(|V|, |E|) = O(|E| \cdot (\log |E| + |V|)) = O(|E| \cdot |V|)$; esto es manteniendo un arreglo de tamaño n para almacenar la clase de cada nodo (IDs). Se puede reducir a $O(|E| \log |V|)$ con una estructura de UNION-FIND basada en árboles AVL, en donde UNION(u, v) y FIND(v) cuestan $O(\log |V|)$.

Lista de Adyacencia

Algunos algoritmos que estudiaremos, reciben como entrada la **lista de adyacencia** Ad de G :



(a)



(b)

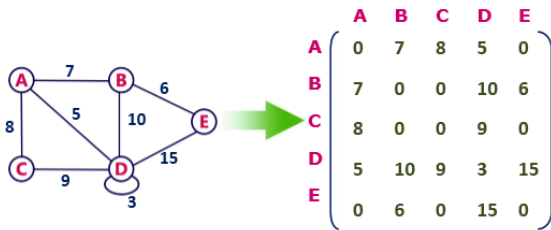
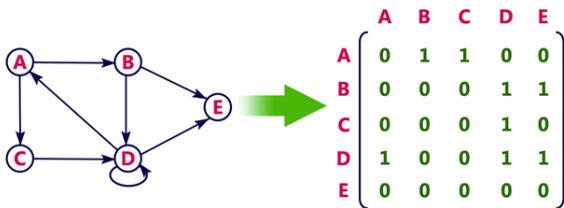
	1	2	3	4	5
1	0	1	0	0	1
2	1	0	1	1	1
3	0	1	0	1	0
4	0	1	1	0	1
5	1	1	0	1	0

(c)

¿Cuáles son los tiempos de búsqueda e inserción en cada caso?

Lista de Adyacencia

Otra forma::



Breadth-First Search Algorithm (BFS)

Dado un grafo $G = (V, E)$ (dirigido o no) y un vértice de partida $s \in V$ (s es llamado *root*), se desea encontrar todos los vértices que se pueden alcanzar desde s y la distancia para llegar a cada uno de ellos (**cantidad mínima de vértices** que se visitan para llegar al destino)

Breadth-First Search Algorithm (BFS)

Dado un grafo $G = (V, E)$ (dirigido o no) y un vértice de partida $s \in V$ (s es llamado *root*), se desea encontrar todos los vértices que se pueden alcanzar desde s y la distancia para llegar a cada uno de ellos (**cantidad mínima de vértices** que se visitan para llegar al destino)

El algoritmo BFS resuelve este problema aplicando la siguiente **estrategía greedy**:

- Comienza desde s (el root) explorando todos los vecinos de s (i.e. sus nodos adyacentes).
- Luego, con cada vecino, prosigue explorando de la misma forma.
- La salida es llamada un BFS Tree de G cuyo root es s .

Entiendase “explorar” como viajar por las aristas disponibles buscando alcanzar nuevos destinos.

Breadth-First Search Algorithm (BFS)

```
BST( $G = (V, Ad), s$ ) donde  $Ad$  es la lista de adyacencia de  $G$  {  
  for each  $u \in V - \{s\}$  do{ //utilizamos una estructura de largo |V|  
     $u.col = WHITE$            //color  
     $u.dist = \infty$           //distancia  
     $u.anc = NULL$            //ancestro  
  }  
   $s.col = GRAY$   
   $s.dist = 0$   
   $s.anc = NULL$   
   $Q = \emptyset$                 //Q es una cola  
  ENQUEUE( $Q, s$ )  
  while ( $Q \neq \emptyset$ ) do {  
     $u = DEQUEUE(Q)$   
    for each  $v \in G.Ad[u]$  do{  
      if( $v.col == WHITE$ ) then{  
         $v.col = GRAY$   
         $v.dist = 1 + u.dist$   
         $v.anc = u$   
        ENQUEUE( $Q, v$ )  
      }  
    }  
     $u.col = BLACK$   
  }  
}
```

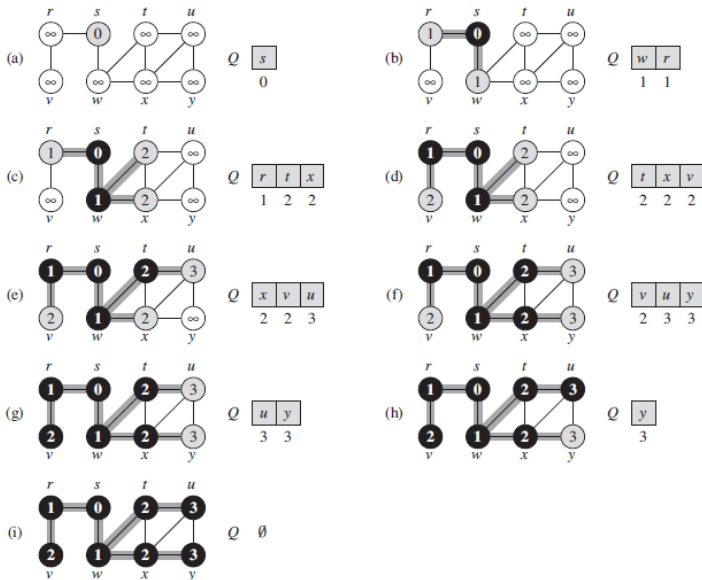
Total running time?

Breadth-First Search Algorithm (BFS)

```
BST( $G = (V, Ad), s$ ) donde  $Ad$  es la lista de adyacencia de  $G$  {  
  for each  $u \in V - \{s\}$  do{ //utilizamos una estructura de largo |V|  
     $u.col = WHITE$            //color  
     $u.dist = \infty$           //distancia  
     $u.anc = NULL$            //ancestro  
  }  
   $s.col = GRAY$   
   $s.dist = 0$   
   $s.anc = NULL$   
   $Q = \emptyset$                //Q es una cola  
  ENQUEUE( $Q, s$ )  
  while ( $Q \neq \emptyset$ ) do {  
     $u = DEQUEUE(Q)$   
    for each  $v \in G.Ad[u]$  do{  
      if( $v.col == WHITE$ ) then{  
         $v.col = GRAY$   
         $v.dist = 1 + u.dist$   
         $v.anc = u$   
        ENQUEUE( $Q, v$ )  
      }  
    }  
     $u.col = BLACK$   
  }  
}
```

Total running time? $\Rightarrow O(|V| + |Ad|)$

Algoritmo BFS - Ejemplo



Imprimir Después de (BFS)

Algoritmo para imprimir el camino más corto de s a v (luego de ejecutar $\text{BFS}()$):

```
PRINT-SP( $G, s, v$ ){  
    if( $s == v$ ) then{  
        PRINT( $s$ )  
    }  
    else if( $v.anc == \text{NULL}$ ) then  
        PRINT("sin camino")  
    else{  
        PRINT-SP( $G, s, v.anc$ )  
        PRINT( $v$ )  
    }  
}
```